Python Practical Programs (Networking & GUI)

Program 1: Create a Socket

Note: Simple socket creation and inspection.

Code (01_socket.py)

```
# Program 1: Create a Socket
# (Original: 45)

import socket

# Create a TCP/IP socket
s = socket.socket(socket.AF_INET, socket.SOCK_STREAM)
print("Socket created successfully")
print("Socket family :", s.family)
print("Socket type   :", s.type)
print("Socket protocol:", s.proto)
s.close()
print("Socket closed")
```

Sample / Expected Output:

```
Socket created successfully
Socket family : 2
Socket type   : 1
Socket protocol: 0
Socket closed
```

Program 2: Identify IP Address

Note: Gets local hostname and IP address.

Code (02_ip_address.py)

```
# Program 2: Identify IP Address
# (Original: 46)

import socket

hostname = socket.gethostname()
ip_address = socket.gethostbyname(hostname)

print("Hostname     :", hostname)
print("IP Address   :", ip_address)

# Also show public IP style lookup for a known host
print("\nGoogle DNS IP:", socket.gethostbyname("dns.google"))
```

Sample / Expected Output:

```
Hostname     : <your-computer-name>
IP Address   : <your-local-ip>

Google DNS IP: 8.8.8.8
```

Program 3: Download Source Code of a Web Page

Note: Fetches and prints the HTML source of a webpage.

Code (03_download_source.py)

```
# Program 3: Download source code of a web page
# (Original: 47)

import urllib.request

url = "https://www.example.com"
try:
    response = urllib.request.urlopen(url)
    source = response.read().decode("utf-8")
    print("Source code of", url)
    print("-" * 50)
    print(source[:1500])          # print first 1500 characters
    print("\n... (truncated)")
except Exception as e:
    print("Error:", e)
```

Sample / Expected Output:

```
Source code of https://www.example.com
--------------------------------------------------
<!doctype html><html lang="en">...
... (truncated)
```

Program 4: Download a Web Page from Internet

Note: Downloads the entire webpage and saves it as an HTML file.

Code (04_download_webpage.py)

```
# Program 4: Download a web page from internet
# (Original: 48)

import urllib.request

url = "https://www.example.com"
filename = "downloaded_page.html"

try:
    urllib.request.urlretrieve(url, filename)
    print("Web page downloaded successfully as:", filename)

    with open(filename, "r", encoding="utf-8") as f:
        content = f.read()
    print("\nFirst 500 characters of the downloaded page:")
    print("-" * 50)
    print(content[:500])
except Exception as e:
    print("Error:", e)
```

Sample / Expected Output:

```
Web page downloaded successfully as: downloaded_page.html

First 500 characters of the downloaded page:
--------------------------------------------------
<!doctype html><html lang="en">...
```

Program 5: Download an Image from Internet

Note: Downloads a PNG image (Python logo) and shows file size.

Code (05_download_image.py)

```
# Program 5: Download an image from internet
# (Original: 49)

import urllib.request

# A small public domain image
url = "https://www.python.org/static/community_logos/python-logo-master-v3-TM.png"
filename = "python_logo.png"

try:
    urllib.request.urlretrieve(url, filename)
    print("Image downloaded successfully as:", filename)

    import os
    size = os.path.getsize(filename)
    print("File size:", size, "bytes")
except Exception as e:
    print("Error:", e)
```

Sample / Expected Output:

```
Image downloaded successfully as: python_logo.png
File size: <size> bytes
```

Program 6: TCP/IP Server and Client

Note: Run the SERVER first in one IDLE/terminal window, then run the CLIENT in another window.

Server Code (06_tcp_server.py)

```
# Program 6: TCP/IP Server
# (Original: 50) - Run this first

import socket

HOST = "127.0.0.1"
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print(f"TCP Server listening on {HOST}:{PORT}")
    conn, addr = s.accept()
    with conn:
        print("Connected by", addr)
        data = conn.recv(1024)
        print("Received from client:", data.decode())
        conn.sendall(b"Hello from TCP Server")
        print("Reply sent to client")
```

Client Code (06_tcp_client.py)

```
# Program 6: TCP/IP Client
# (Original: 50) - Run this after starting the server

import socket

HOST = "127.0.0.1"
PORT = 65432

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    s.sendall(b"Hello from TCP Client")
    data = s.recv(1024)
    print("Received from server:", data.decode())
```

Sample / Expected Output:

```
--- Server Output ---
TCP Server listening on 127.0.0.1:65432
Connected by ('127.0.0.1', xxxxx)
```

```
Received from client: Hello from TCP Client
Reply sent to client

--- Client Output ---
Received from server: Hello from TCP Server
```

Program 7: Two-way Communication (Server to Client and Client to Server)

Note: Run SERVER first. Both sides can send messages. Type 'exit' to quit.

Server Code (07_twoway_server.py)

```
# Program 7: Two-way communication - Server
# (Original: 51) - Run this first

import socket

HOST = "127.0.0.1"
PORT = 65433

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print(f"Two-way Server listening on {HOST}:{PORT}")
    conn, addr = s.accept()
    with conn:
        print("Connected by", addr)
        while True:
            data = conn.recv(1024)
            if not data or data.decode().lower() == "exit":
                print("Client disconnected")
                break
            print("Client:", data.decode())
            reply = input("Server: ")
            conn.sendall(reply.encode())
            if reply.lower() == "exit":
                break
```

Client Code (07_twoway_client.py)

```
# Program 7: Two-way communication - Client
# (Original: 51) - Run this after starting the server

import socket

HOST = "127.0.0.1"
PORT = 65433

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    print("Connected to server. Type 'exit' to quit.")
    while True:
        msg = input("Client: ")
        s.sendall(msg.encode())
```

```
        if msg.lower() == "exit":
            break
        data = s.recv(1024)
        print("Server:", data.decode())
        if data.decode().lower() == "exit":
            break
```

Sample / Expected Output:

```
--- Server ---
Two-way Server listening on 127.0.0.1:65433
Connected by ('127.0.0.1', xxxxx)
Client: Hi Server
Server: Hello Client
Client: How are you?
Server: Fine, thanks

--- Client ---
Connected to server. Type 'exit' to quit.
Client: Hi Server
Server: Hello Client
Client: How are you?
Server: Fine, thanks
```

Program 8: UDP Server and Client

Note: Run the UDP SERVER first, then the CLIENT. UDP is connectionless.

Server Code (08_udp_server.py)

```
# Program 8: UDP Server
# (Original: 52) - Run this first

import socket

HOST = "127.0.0.1"
PORT = 65434

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.bind((HOST, PORT))
    print(f"UDP Server listening on {HOST}:{PORT}")
    data, addr = s.recvfrom(1024)
    print("Received from", addr, ":", data.decode())
    s.sendto(b"Hello from UDP Server", addr)
    print("Reply sent")
```

Client Code (08_udp_client.py)

```
# Program 8: UDP Client
# (Original: 52) - Run this after starting the server

import socket

HOST = "127.0.0.1"
PORT = 65434

with socket.socket(socket.AF_INET, socket.SOCK_DGRAM) as s:
    s.sendto(b"Hello from UDP Client", (HOST, PORT))
    data, addr = s.recvfrom(1024)
    print("Received from server:", data.decode())
```

Sample / Expected Output:

```
--- Server Output ---
UDP Server listening on 127.0.0.1:65434
Received from ('127.0.0.1', xxxxx) : Hello from UDP Client
Reply sent

--- Client Output ---
Received from server: Hello from UDP Server
```

Program 9: File Server and Client

Note: Place a file named sample.txt in the same folder as the server. Run SERVER first, then CLIENT. The client receives the file as received_sample.txt.

Server Code (09_file_server.py)

```
# Program 9: File Server
# (Original: 53) - Run this first
# Place a file named "sample.txt" in the same folder

import socket

HOST = "127.0.0.1"
PORT = 65435

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.bind((HOST, PORT))
    s.listen()
    print(f"File Server listening on {HOST}:{PORT}")
    conn, addr = s.accept()
    with conn:
        print("Connected by", addr)
        filename = conn.recv(1024).decode()
        print("Client requested file:", filename)
        try:
            with open(filename, "rb") as f:
                data = f.read()
            conn.sendall(data)
            print("File sent successfully")
        except FileNotFoundError:
            conn.sendall(b"ERROR: File not found")
            print("File not found")
```

Client Code (09_file_client.py)

```
# Program 9: File Client
# (Original: 53) - Run this after starting the server
# Make sure sample.txt exists on the server side

import socket

HOST = "127.0.0.1"
PORT = 65435

with socket.socket(socket.AF_INET, socket.SOCK_STREAM) as s:
    s.connect((HOST, PORT))
    filename = "sample.txt"
```

```
    s.sendall(filename.encode())
    data = s.recv(4096)
    if data.startswith(b"ERROR"):
        print(data.decode())
    else:
        with open("received_" + filename, "wb") as f:
            f.write(data)
        print("File received and saved as received_" + filename)
        print("Content:")
        print(data.decode())
```

Sample / Expected Output:

```
--- Server ---
File Server listening on 127.0.0.1:65435
Connected by ('127.0.0.1', xxxxx)
Client requested file: sample.txt
File sent successfully

--- Client ---
File received and saved as received_sample.txt
Content:
This is a sample file for the File Server program.
Hello from Python networking practical!
```

Program 10: Sending an Email

Note: Update sender, receiver and password. For Gmail: enable 2-Step Verification and create an App Password. Do not use your normal password.

Code (10_send_email.py)

```
# Program 10: Sending an Email
# (Original: 54)
# Note: Replace with your own email and App Password.
# For Gmail enable 2FA and create an App Password.

import smtplib
from email.mime.text import MIMEText
from email.mime.multipart import MIMEMultipart

sender = "your_email@gmail.com"
receiver = "receiver_email@gmail.com"
password = "your_app_password"   # Use App Password, not normal password

msg = MIMEMultipart()
msg["From"] = sender
msg["To"] = receiver
msg["Subject"] = "Test Email from Python"

body = "Hello,\nThis is a test email sent using Python.\n\nRegards"
msg.attach(MIMEText(body, "plain"))

try:
    server = smtplib.SMTP("smtp.gmail.com", 587)
    server.starttls()
    server.login(sender, password)
    server.sendmail(sender, receiver, msg.as_string())
    server.quit()
    print("Email sent successfully!")
except Exception as e:
    print("Error while sending email:", e)
    print("\nNote: Update sender, receiver and password with valid credentials.")
    print("For Gmail use an App Password after enabling 2-Step Verification.")
```

Sample / Expected Output:

```
Email sent successfully!

(OR if credentials are not set)
Error while sending email: ...
Note: Update sender, receiver and password with valid credentials.
For Gmail use an App Password after enabling 2-Step Verification.
```

Program 11: GUI with Buttons, Labels and Entry Fields

Note: A simple tkinter window. Enter a name and click the button to see a greeting.

Code (11_gui_widgets.py)

```
# Program 11: GUI with Buttons, Labels, Entry fields
# (Original: 55)

import tkinter as tk
from tkinter import messagebox

def greet():
    name = entry.get()
    if name.strip():
        label_result.config(text=f"Hello, {name}!")
    else:
        label_result.config(text="Please enter your name")

root = tk.Tk()
root.title("Simple GUI Demo")
root.geometry("350x200")

label = tk.Label(root, text="Enter your name:", font=("Arial", 12))
label.pack(pady=10)

entry = tk.Entry(root, width=30, font=("Arial", 11))
entry.pack(pady=5)

btn = tk.Button(root, text="Greet Me", command=greet, font=("Arial", 11),
bg="lightblue")
btn.pack(pady=10)

label_result = tk.Label(root, text="", font=("Arial", 12), fg="green")
label_result.pack(pady=10)

root.mainloop()
```

Sample / Expected Output:

```
(GUI Window opens)
Title: Simple GUI Demo
- Label: "Enter your name:"
- Entry field
- Button: "Greet Me"
- Result label shows "Hello, <name>!" after clicking
```

Program 12: Confirmation and Alerts using Dialogs

Note: Demonstrates showinfo, showwarning, showerror, askquestion, askyesno and askokcancel dialogs.

Code (12_dialogs.py)

```
# Program 12: Confirmation and Alerts using Dialogs
# (Original: 56)

import tkinter as tk
from tkinter import messagebox

def show_info():
    messagebox.showinfo("Information", "This is an information message.")

def show_warning():
    messagebox.showwarning("Warning", "This is a warning message!")

def show_error():
    messagebox.showerror("Error", "This is an error message!")

def ask_question():
    result = messagebox.askquestion("Question", "Do you want to continue?")
    label.config(text=f"You selected: {result}")

def ask_yesno():
    result = messagebox.askyesno("Confirm", "Are you sure?")
    label.config(text=f"Yes/No result: {result}")

def ask_okcancel():
    result = messagebox.askokcancel("OK / Cancel", "Proceed with the action?")
    label.config(text=f"OK/Cancel result: {result}")

root = tk.Tk()
root.title("Dialog Boxes Demo")
root.geometry("400x350")

tk.Label(root, text="Click buttons to see different dialogs", font=("Arial",
12)).pack(pady=10)

tk.Button(root, text="Show Info", command=show_info, width=20).pack(pady=5)
tk.Button(root, text="Show Warning", command=show_warning, width=20).pack(pady=5)
tk.Button(root, text="Show Error", command=show_error, width=20).pack(pady=5)
tk.Button(root, text="Ask Question", command=ask_question, width=20).pack(pady=5)
tk.Button(root, text="Ask Yes/No", command=ask_yesno, width=20).pack(pady=5)
tk.Button(root, text="Ask OK/Cancel", command=ask_okcancel, width=20).pack(pady=5)
```

```
label = tk.Label(root, text="", font=("Arial", 11), fg="blue")
label.pack(pady=15)

root.mainloop()
```

Sample / Expected Output:

```
(GUI Window opens with 6 buttons)
Clicking each button opens the corresponding dialog box.
Results of Yes/No / OK-Cancel questions are shown in a label.
```

Program 13: Simple Calculator

Note: A basic calculator GUI using tkinter. Supports +, -, *, / and clear.

Code (13_calculator.py)

```
# Program 13: Simple Calculator
# (Original: 57)

import tkinter as tk

def click(event):
    global expression
    text = event.widget.cget("text")
    if text == "=":
        try:
            result = str(eval(expression))
            entry_var.set(result)
            expression = result
        except Exception:
            entry_var.set("Error")
            expression = ""
    elif text == "C":
        expression = ""
        entry_var.set("")
    else:
        expression += text
        entry_var.set(expression)

root = tk.Tk()
root.title("Simple Calculator")
root.geometry("300x400")
root.resizable(False, False)

expression = ""
entry_var = tk.StringVar()

entry = tk.Entry(root, textvariable=entry_var, font=("Arial", 20),
                 bd=10, relief=tk.RIDGE, justify="right")
entry.pack(fill=tk.BOTH, padx=10, pady=10, ipady=10)

buttons = [
    ["7", "8", "9", "/"],
    ["4", "5", "6", "*"],
    ["1", "2", "3", "-"],
    ["0", "C", "=", "+"]
]
```

```
for row in buttons:
    frame = tk.Frame(root)
    frame.pack(expand=True, fill="both")
    for btn_text in row:
        btn = tk.Button(frame, text=btn_text, font=("Arial", 18), height=2, width=4)
        btn.pack(side=tk.LEFT, expand=True, fill="both", padx=2, pady=2)
        btn.bind("<Button-1>", click)

root.mainloop()
```

Sample / Expected Output:

```
(GUI Calculator Window opens)
Buttons: 0-9, +, -, *, /, =, C
Example: 12 + 5 = 17
Press C to clear.
```